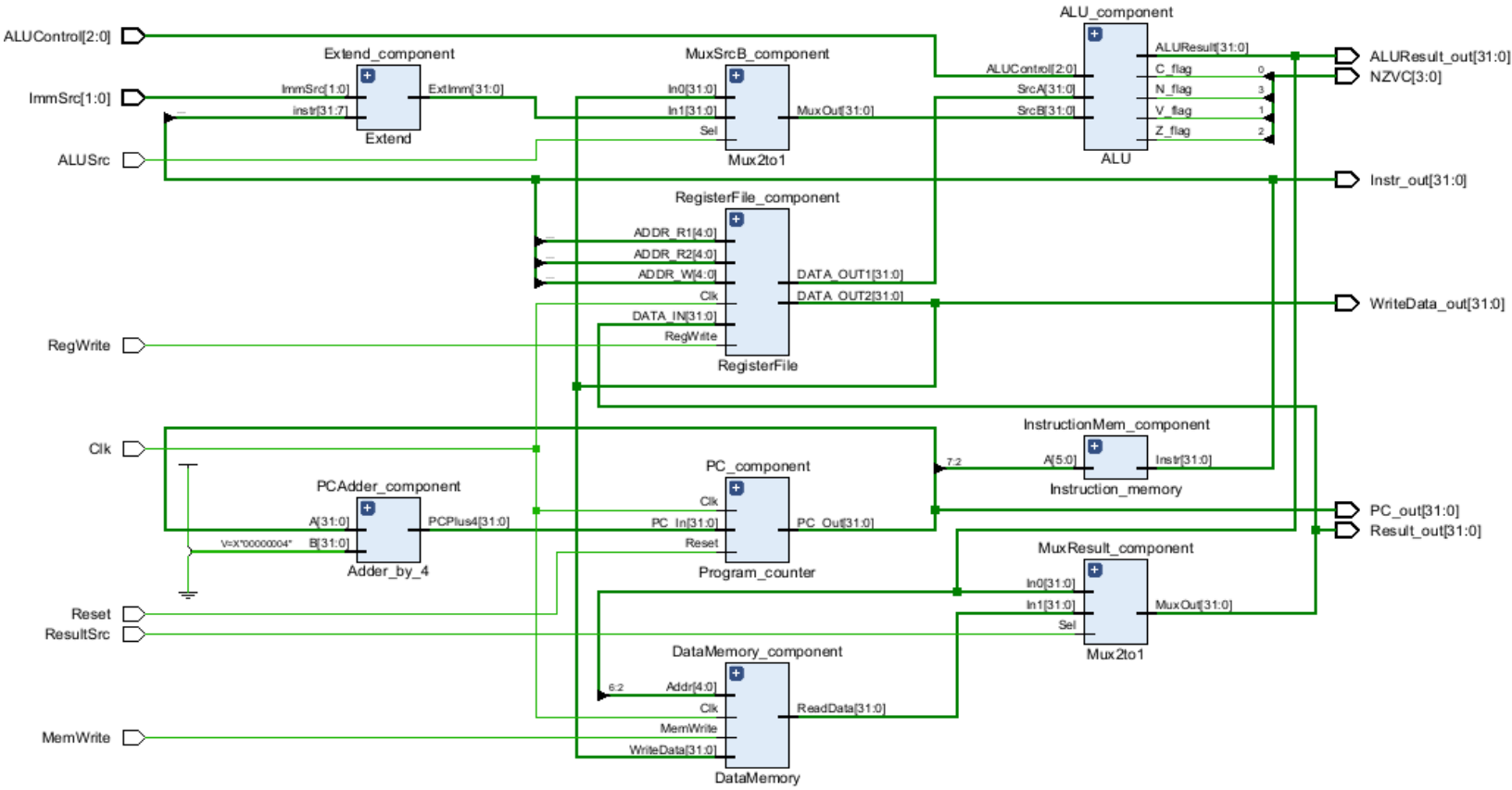
RISC-V Single-Cycle Processor Implementation Report in **VHDL**

7-1. Description of Processor Elements and Structure

7-1.1 Datapath RTL



In the RTL diagram, we can observe the datapath components. Initially, there is the register for the **Program Counter** (PC). The **ROM** memory for storing instructions receives the program counter value and outputs the corresponding instruction. This instruction goes as an input to the register file to identify the source and destination registers, as well as to the Extend unit. The **adder** increments the program counter by 4.

The **register file** receives the instruction fields mentioned above and has two output ports for reading data. It also has one port for writing data, provided the `RegWrite` signal from the control unit permits it. The processor writes to the register file when the instruction is not a Store type.

Next, we have the **sign extension component,** which performs the necessary extensions for I-type and Store instructions. The extended operand serves as the input to port '1' of the multiplexer for the ALU's `SrcB`. Depending on the `ALUSrc` signal, this multiplexer passes either the extended value (for Immediate, Load, and Store instructions) or the value of the second source register (for R-type instructions) as the second input to the ALU.

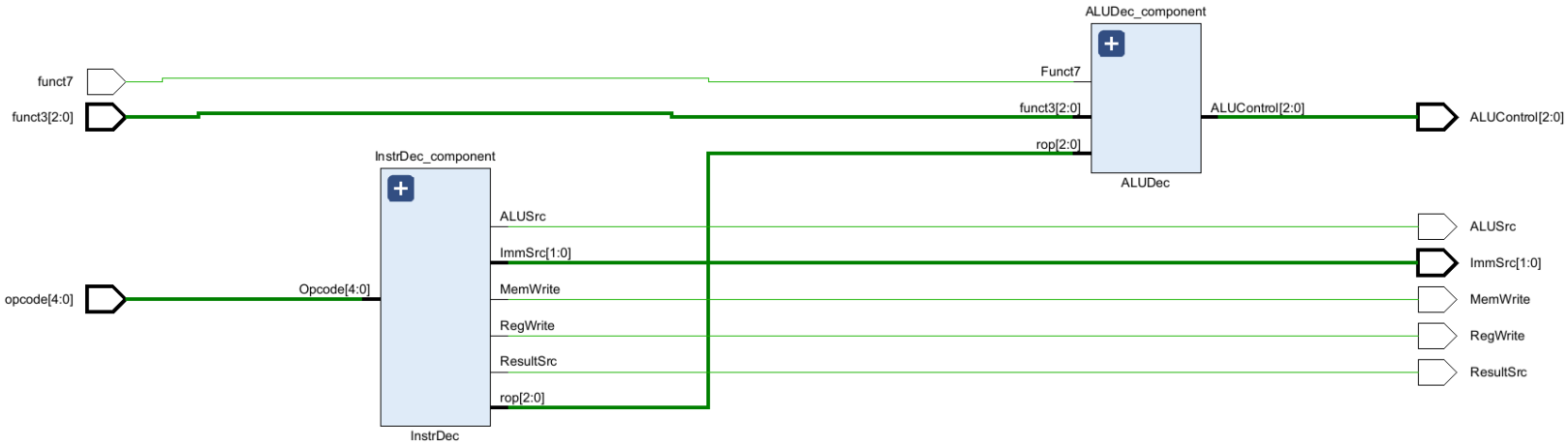
The ALU component receives the two inputs `srcA` and `srcB`, along with the `ALUControl` signal from the control unit, which dictates the operation to be performed. It outputs the `ALUResult` signal, which is stored in the register file for R-type and I-type instructions. When the instruction is LOAD, the ALU result acts as a memory address to read the value from memory, which is ultimately written back to the register file. In the case of a STORE instruction, the `ALUResult` is an address; nothing is saved to the register file, but the content of `RD2` from the register file is stored at this memory address.

The `SLTU` or `SLT` signal was disabled because comparison instructions are not included in the requested instruction set, as stated at the end of the assignment instructions (unnecessary signals should not be implemented).

The **RAM** used as data memory writes data to the `ALUResult` address during a Store operation and provides a read port for Load operations. Finally, there is a **multiplexer** to select the source of the data to be written back to the register file based on the `ResultSrc` signal (from RAM or ALU). For SW instructions, where we do not want to write back to a register, the Control Unit ensures that `RegWrite` is set to 0.

Components 2-5.2 and 2-5.3 were NOT implemented as they relate to branch instructions. As specified at the end of the assignment, even though they are mentioned, they should not be implemented. The program counter is always updated to PC+4.

7-1.2 Control Unit RTL Diagram



In the RTL diagram of the Control Unit, we observe that only the `InstrDec` and `ALUDec` sub-modules were implemented. The `PCLogic` sub-module (and the corresponding 2-5.2, 2-5.3 circuits) was not implemented, as the required instruction set does not include Branch instructions. Therefore, the selection of the next address does not depend on conditions, and the `PCSrc` signal would have a constant value of '0' (selecting PC+4), making the circuit redundant.

The operation of the Control Unit is divided into:

1. **InstrDec:** Receives the Opcode of the instruction and generates the main control signals for the datapath (`ResultSrc`, `MemWrite`, `ALUSrc`, `ImmSrc`, `RegWrite`). Simultaneously, it generates the internal `rop` signal, which informs the `ALUDEC` about the type of operation.
2. **ALUDec (ALU Decoder):** Receives the `rop` signal as well as the `funct3` and `funct7` fields of the instruction. Based on these, it determines the exact operation the ALU will execute and generates the `ALUControl` signal.

Modifications to the main truth table 4-2.1: The signals and logic for the SLTU and SLT comparison instructions were removed, and XOR is only performed when `ALUControl = 110`. For LW and SW instructions, the ALUDEC generates `ALUControl = "000"` (Addition) so that the ALU calculates the memory address (Base Register + Offset).

Truth Table for InstrDec:

Op(6:2)	Rop(2:0)	ResultSrc	MemWrite	ALUsrc	ImmSrc(1:0)	RegWrite	Type
00000	000	1	0	1	00	1	*LW*
01100	010	0	0	0	00	1	*R*
00100	011	0	0	1	00	1	*I*
01000	001	0	1	1	01	0	*SW*

Truth Table for ALUDec:

Rop(2:0)	Funct3(2:0)	Func7	ALUControl(2:0)	Instruction
000	x	x	000	LW (ADD στην ALU
001	x	x	000	SW (ADD στην ALU)
010	000	0	000	ADD
010	000	1	001	SUB
010	100	x	110	XOR
010	110	x	101	OR
010	111	x	100	AND
011	000	x	000	ADDI
011	100	x	110	XORI
011	110	X	101	ORI
011	111	X	100	ANDI

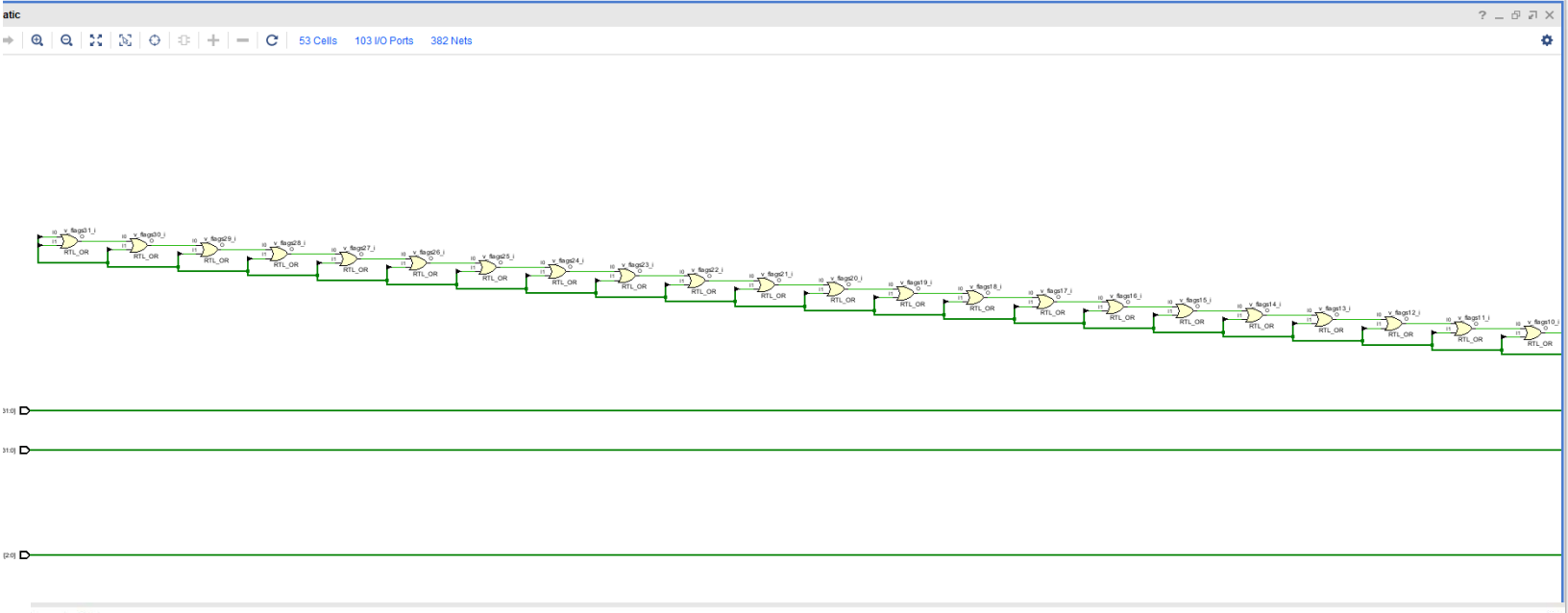
Deductions from the tables:

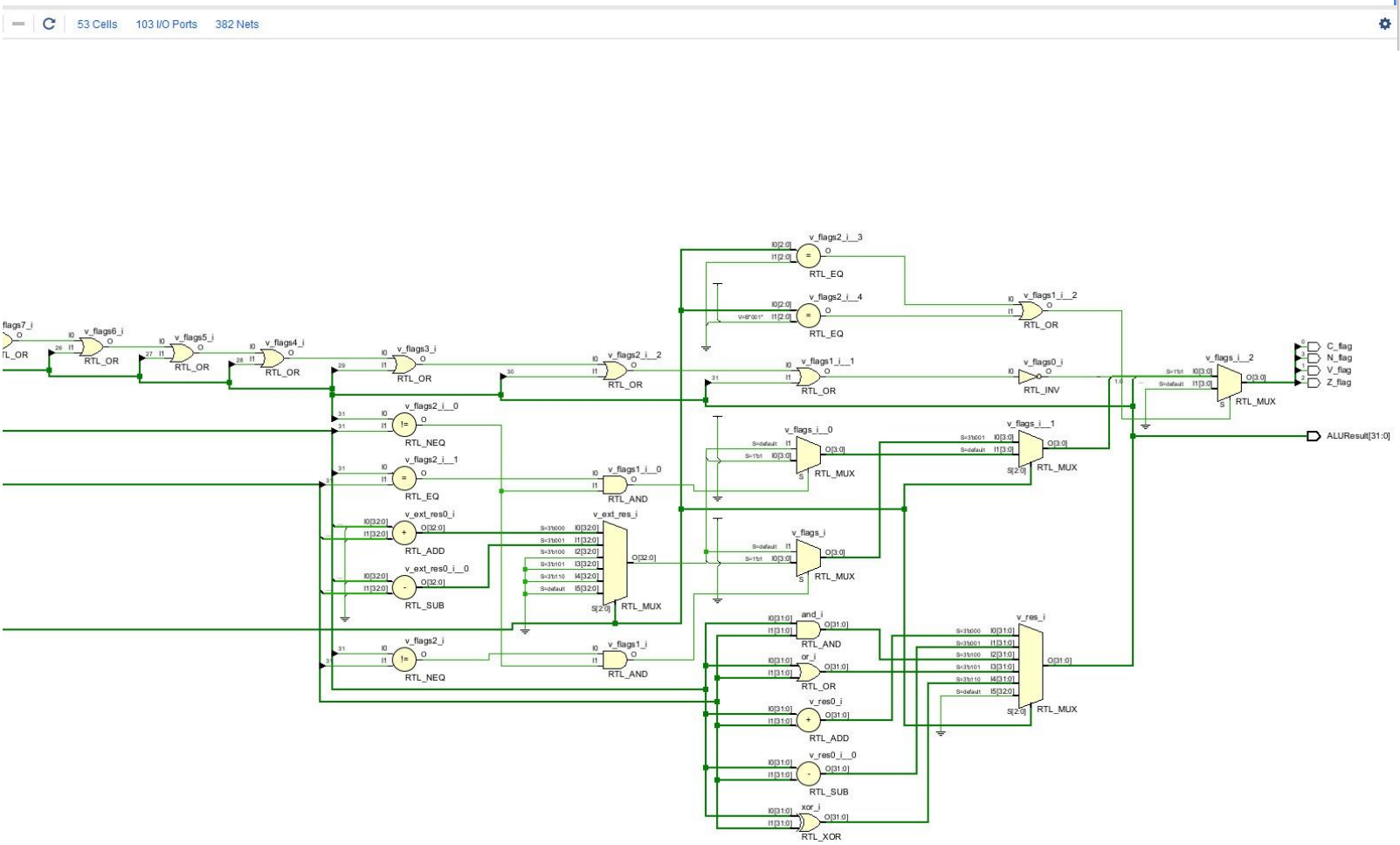
1. **LW Instruction:** In `InstrDec`, `ALUSrc` is set to 1 so the ALU adds the base register and the Immediate. `ResultSrc` is 1 to route data from memory (and not the ALU) back to the destination register. `RegWrite` is set to 1. In `ALUDec`, `ALUControl = 000` (Addition) is generated to calculate the address.

2. **SW Instruction:** In `InstrDec`, `MemWrite` is 1 to write to memory. `RegWrite` is 0 to protect register contents. `ALUSrc` is 1 (address calculation). `ImmSrc` is set to 01 for proper extension. In `ALUDec`, `ALUControl = 000` (Addition) is also generated.
3. **R-Type Instructions:** In `InstrDec`, `ALUSrc` is '0' so the ALU processes the two registers (RD1, RD2). `ResultSrc` is '0' (ALU result to register). `RegWrite = 1` to write the result back to the register. In `ALUDec`, the instruction is differentiated based on the `funct3` and `funct7` fields.
4. **I-Type Instructions:** In `InstrDec`, they behave similarly to R-Type regarding writing (`RegWrite=1`), but `ALUSrc` is '1' to use the Immediate. In `ALUDec`, the operation is determined by `funct3`.

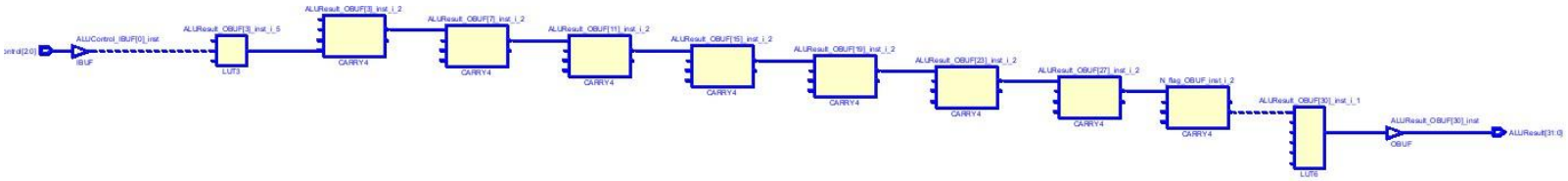
7-1.3 ALU RTL Diagram

The RTL diagram consists of many NOR gates connected sequentially because the assignment explicitly stated that the Z (zero) flag must be implemented using NOR gates. I tested implementing it with a simple IF statement; in the elaborated design, a multiplexer was created, but in the synthesized design, the resources used were identical, so there was no difference. It also consists of all other necessary elements for the ALU (adders, multiplexers, logic gates for results). It accepts two operands and generates the `ALUResult` as well as the NZVC flags. Its operation is determined by the `ALUControl` arriving from the control unit.


atic
53 Cells
103 I/O Ports
382 Nets
RTL_OR


53 Cells
103 I/O Ports
382 Nets
v_flags7_i
v_flags6_i
v_flags5_i
v_flags4_i
v_flags3_i
v_flags2_i__2
v_flags2_i__3
v_flags2_i__4
v_flags1_i__2
v_flags1_i__1
v_flags0_i
v_flags2_i__0
v_flags2_i__1
v_flags1_i__0
v_ext_res0_i
v_ext_res_i
v_ext_res0_i__0
v_flags2_i
v_flags1_i
v_flags_i__0
v_flags_i__1
v_flags_i
v_flags_i__2
and_i
or_i
v_res_i
v_res0_i
v_res0_i__0
xor_i
RTL_OR
RTL_EQ
RTL_INV
RTL_MUX
RTL_NEQ
RTL_AND
RTL_ADD
RTL_SUB
RTL_XOR
C_flag
N_flag
V_flag
Z_flag
ALUResult[31:0]

Propagation Delay Path 11: 15.786 ns -> Frequency 63.347 MHz

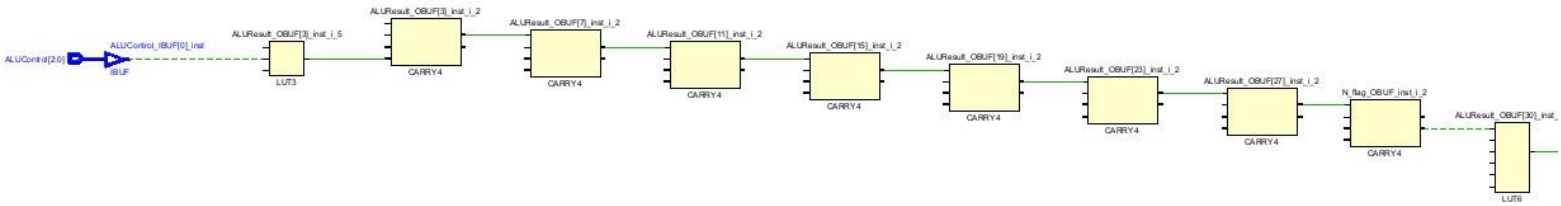


Design Runs | DRC | Power | Timing | Utilization

Timing Checks - Setup

Name	Slack ^1	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Logic %	Net %	Requirement	Source Clock	Destination Clock	Exception	Skew
Unconstrained Paths (1)																
(none) (10)																
Path 11	∞	12	3	98	ALUControl[0]	ALUResult[30]	15.786	5.555	10.231	35.2	64.8	∞	input port clock			
Path 12	∞	7	3	98	ALUControl[0]	ALUResult[9]	15.401	4.974	10.427	32.3	67.7	∞	input port clock			
Path 13	∞	11	3	98	ALUControl[0]	ALUResult[26]	15.373	5.384	9.989	35.0	65.0	∞	input port clock			
Path 14	∞	12	3	98	ALUControl[0]	ALUResult[28]	15.368	5.510	9.858	35.9	64.1	∞	input port clock			
Path 15	∞	9	3	98	ALUControl[0]	ALUResult[17]	15.356	5.243	10.112	34.1	65.9	∞	input port clock			
Path 16	∞	11	3	98	ALUControl[0]	ALUResult[24]	15.277	5.358	9.919	35.1	64.9	∞	input port clock			
Path 17	∞	10	3	98	ALUControl[0]	ALUResult[22]	15.264	5.282	9.983	34.6	65.4	∞	input port clock			
Path 18	∞	10	3	98	ALUControl[0]	ALUResult[20]	15.194	5.264	9.930	34.6	65.4	∞	input port clock			
Path 19	∞	6	3	98	ALUControl[0]	ALUResult[5]	15.147	4.852	10.295	32.0	68.0	∞	input port clock			
Path 20	∞	6	3	98	ALUControl[0]	ALUResult[7]	15.114	4.842	10.273	32.0	68.0	∞	input port clock			

Contamination Delay Path 1: 2.079 ns



Reports | Design Runs | DRC | Power | Timing | Utilization

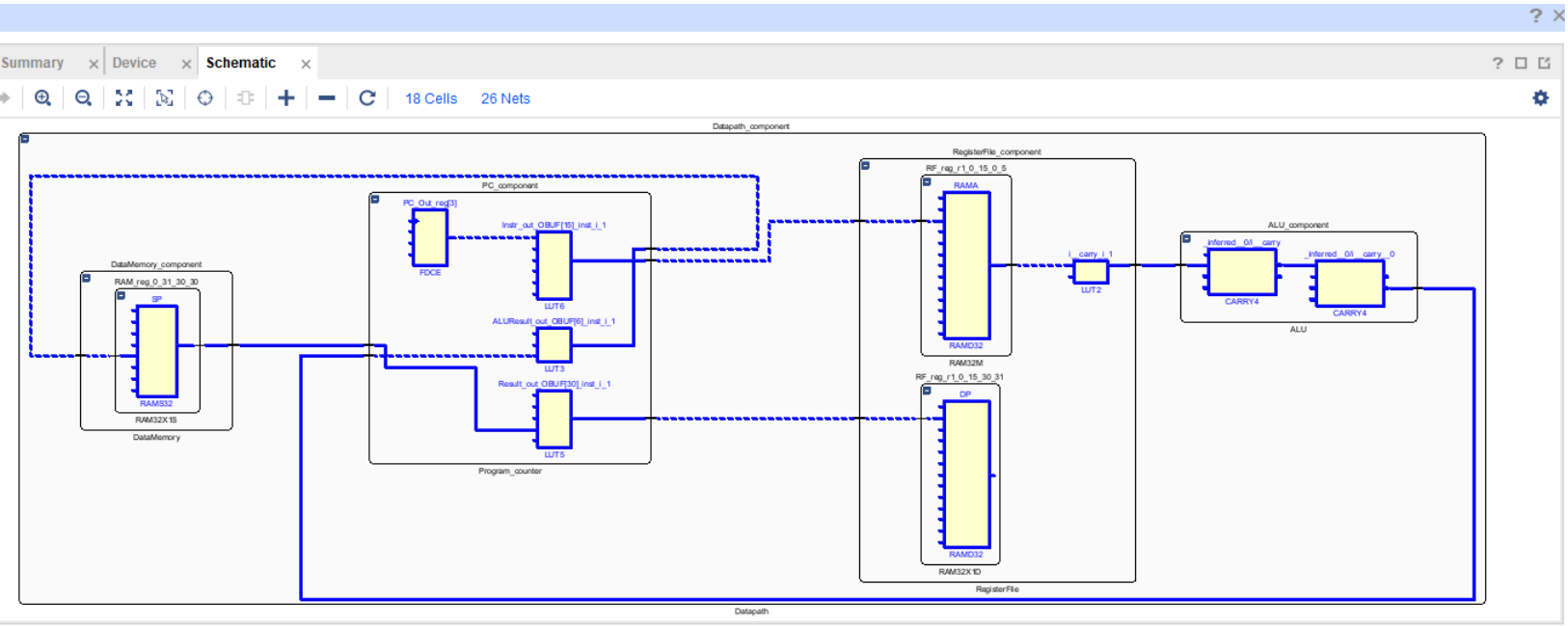
Timing Checks - Hold

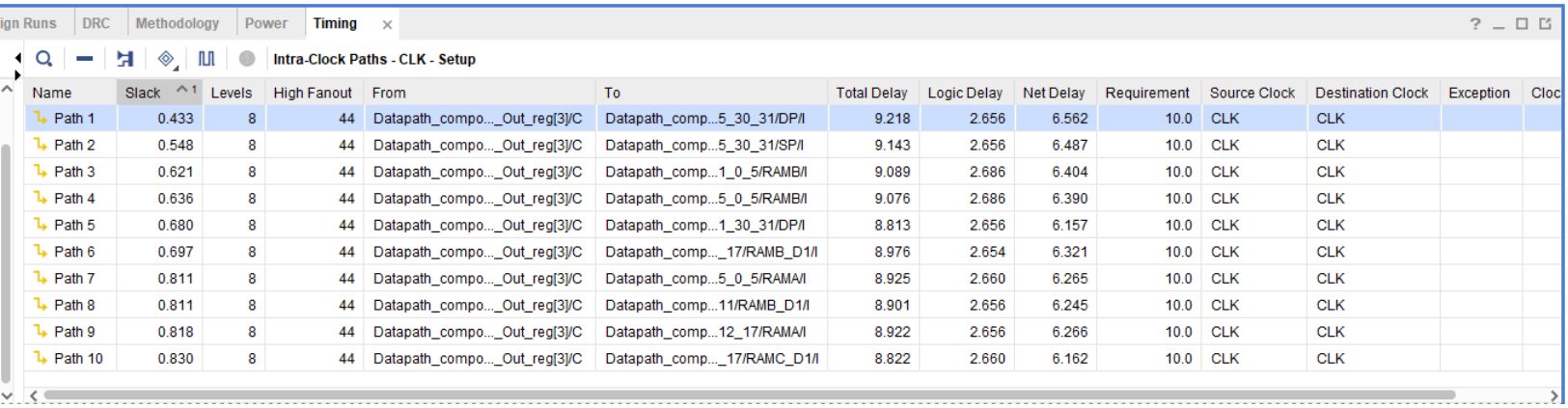
Name	Slack ^1	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Logic %	Net %	Requirement	Source Clock	Destinat
Unconstrained Paths (1)														
(none) (10)														
Path 1	∞	3	2	98	ALUControl[0]	V_flag	2.079	1.356	0.723	65.2	34.8	-∞	input port clock	
Path 2	∞	3	2	36	ALUControl[1]	N_flag	2.161	1.419	0.742	65.7	34.3	-∞	input port clock	
Path 3	∞	6	4	3	SrcB[18]	Z_flag	2.996	1.577	1.419	52.6	47.4	-∞	input port clock	
Path 4	∞	3	2	36	ALUControl[1]	C_flag	3.072	1.491	1.581	48.5	51.5	-∞	input port clock	
Path 5	∞	3	2	4	SrcB[31]	ALUResult[31]	3.339	1.418	1.921	42.5	57.5	-∞	input port clock	
Path 6	∞	3	2	3	SrcB[15]	ALUResult[15]	3.352	1.391	1.961	41.5	58.5	-∞	input port clock	
Path 7	∞	3	2	3	SrcB[29]	ALUResult[29]	3.361	1.455	1.906	43.3	56.7	-∞	input port clock	
Path 8	∞	3	2	3	SrcB[27]	ALUResult[27]	3.386	1.427	1.959	42.1	57.9	-∞	input port clock	
Path 9	∞	3	2	3	SrcB[21]	ALUResult[21]	3.490	1.432	2.058	41.0	59.0	-∞	input port clock	
Path 10	∞	3	2	3	SrcB[2]	ALUResult[2]	3.497	1.356	2.141	38.8	61.2	-∞	input port clock	

Looking at the propagation delay of the standalone ALU, we see that it is quite large. Later on, we will see that the processor's frequency is higher, even though much logic is added. This occurs because Vivado connects the ports to physical pins on the FPGA, meaning the wires travel long distances. However, when the ALU is integrated into the processor's datapath, its communication with other components (e.g., Register File) is done exclusively through internal routing.

7-1.4 Overall Processor Timing

Worst Setup Critical Path (Path 1): 9.218 ns





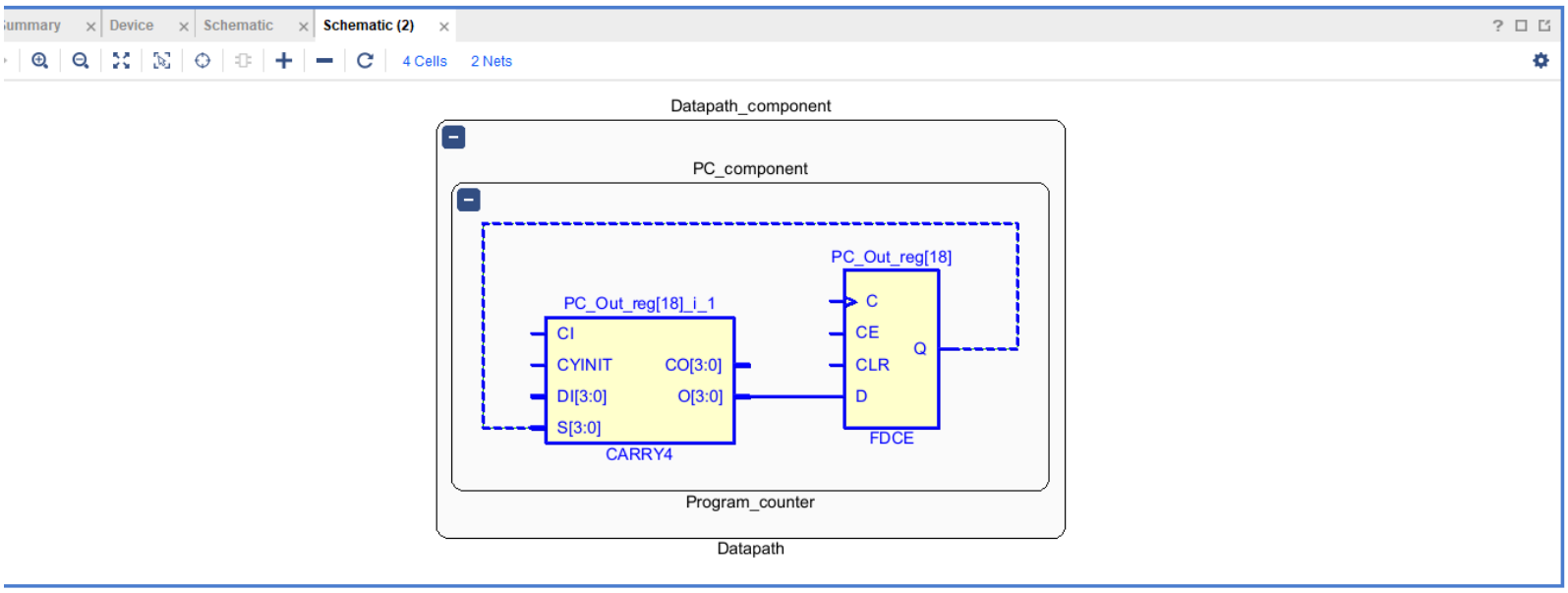
ign Runs | DRC | Methodology | Power | Timing

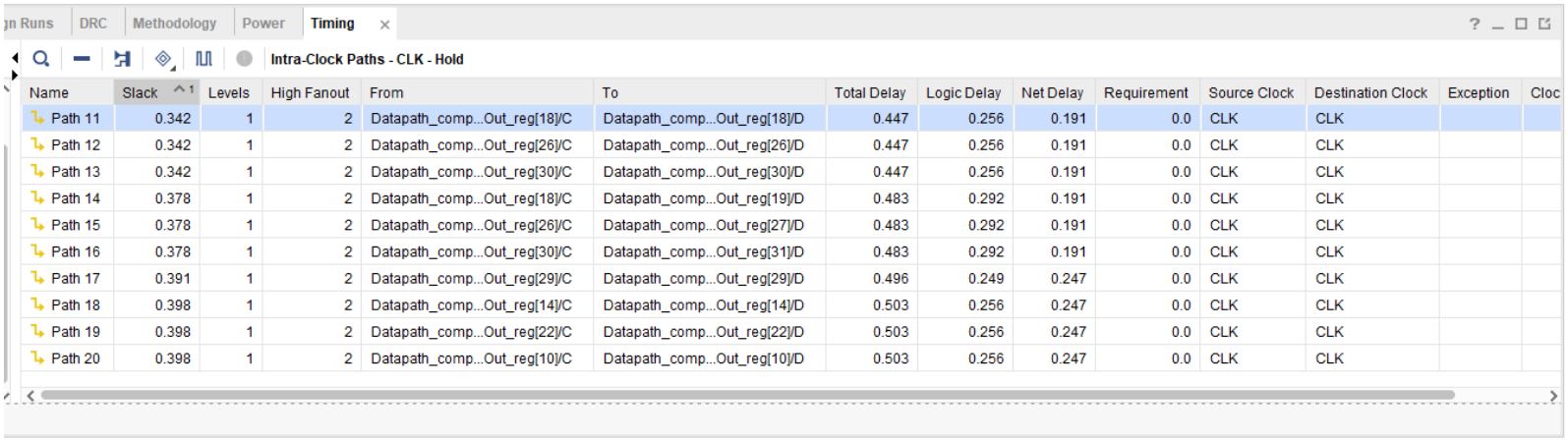
Intra-Clock Paths - CLK - Setup

Name	Slack ^1	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Cloc
Path 1	0.433	8	44	Datapath_compo..._Out_reg[3]/C	Datapath_comp...5_30_31/DP/I	9.218	2.656	6.562	10.0	CLK	CLK		
Path 2	0.548	8	44	Datapath_compo..._Out_reg[3]/C	Datapath_comp...5_30_31/SP/I	9.143	2.656	6.487	10.0	CLK	CLK		
Path 3	0.621	8	44	Datapath_compo..._Out_reg[3]/C	Datapath_comp...1_0_5/RAMB/I	9.089	2.686	6.404	10.0	CLK	CLK		
Path 4	0.636	8	44	Datapath_compo..._Out_reg[3]/C	Datapath_comp...5_0_5/RAMB/I	9.076	2.686	6.390	10.0	CLK	CLK		
Path 5	0.680	8	44	Datapath_compo..._Out_reg[3]/C	Datapath_comp...1_30_31/DP/I	8.813	2.656	6.157	10.0	CLK	CLK		
Path 6	0.697	8	44	Datapath_compo..._Out_reg[3]/C	Datapath_comp..._17/RAMB_D1/I	8.976	2.654	6.321	10.0	CLK	CLK		
Path 7	0.811	8	44	Datapath_compo..._Out_reg[3]/C	Datapath_comp...5_0_5/RAMA/I	8.925	2.660	6.265	10.0	CLK	CLK		
Path 8	0.811	8	44	Datapath_compo..._Out_reg[3]/C	Datapath_comp...11/RAMB_D1/I	8.901	2.656	6.245	10.0	CLK	CLK		
Path 9	0.818	8	44	Datapath_compo..._Out_reg[3]/C	Datapath_comp...12_17/RAMA/I	8.922	2.656	6.266	10.0	CLK	CLK		
Path 10	0.830	8	44	Datapath_compo..._Out_reg[3]/C	Datapath_comp..._17/RAMC_D1/I	8.822	2.660	6.162	10.0	CLK	CLK		

The maximum operating frequency of the processor is derived when the slack of the worst critical path becomes 0. Therefore, the minimum period = 10 - 0.433 = 9.567 ns, resulting in a maximum frequency of 1 / 9.567 = 104.535 MHz.

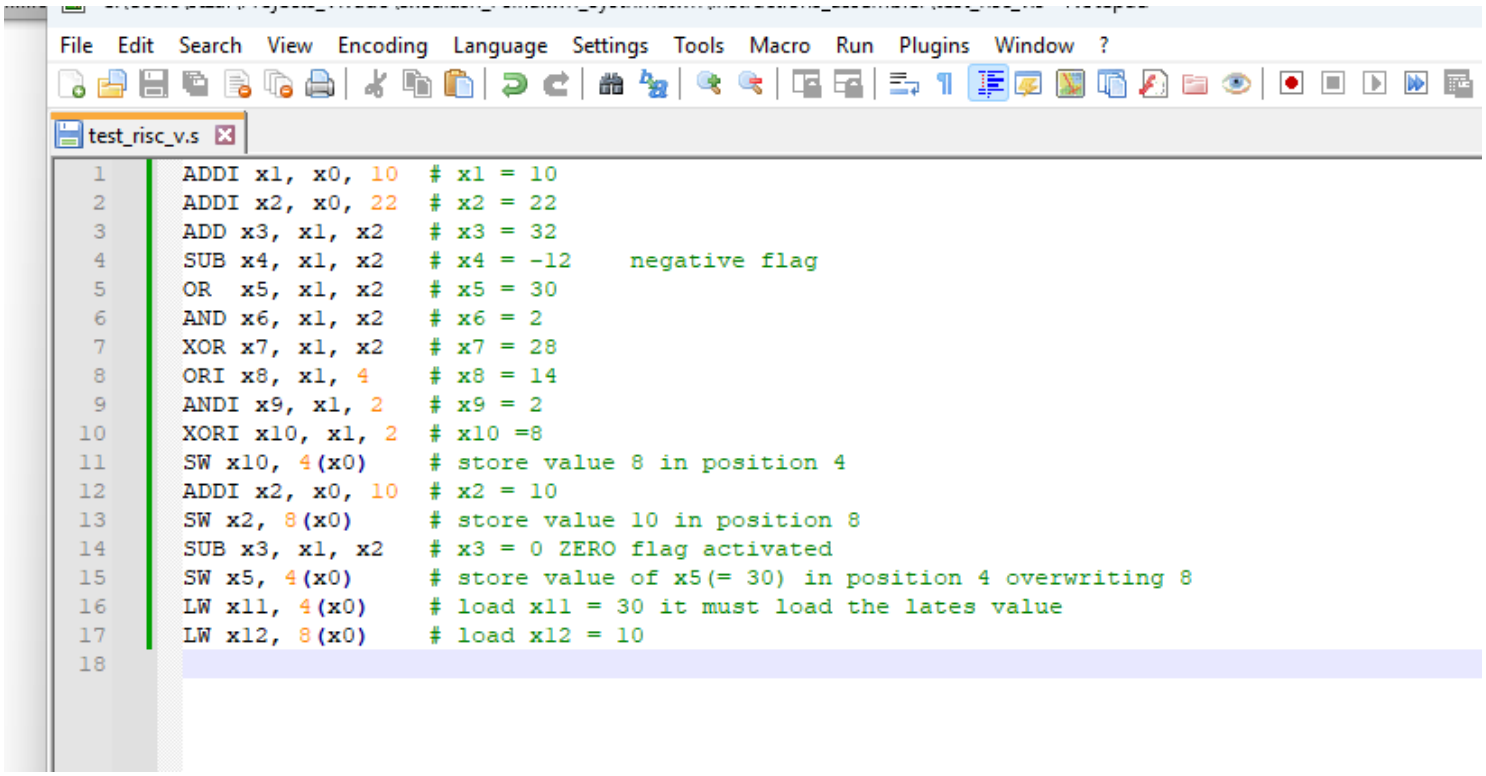
Worst Hold Short Path (Path 11): 0.447 ns





Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Cloc
Path 11	0.342	1	2	Datapath_comp...Out_reg[18]/C	Datapath_comp...Out_reg[18]/D	0.447	0.256	0.191	0.0	CLK	CLK		
Path 12	0.342	1	2	Datapath_comp...Out_reg[26]/C	Datapath_comp...Out_reg[26]/D	0.447	0.256	0.191	0.0	CLK	CLK		
Path 13	0.342	1	2	Datapath_comp...Out_reg[30]/C	Datapath_comp...Out_reg[30]/D	0.447	0.256	0.191	0.0	CLK	CLK		
Path 14	0.378	1	2	Datapath_comp...Out_reg[18]/C	Datapath_comp...Out_reg[19]/D	0.483	0.292	0.191	0.0	CLK	CLK		
Path 15	0.378	1	2	Datapath_comp...Out_reg[26]/C	Datapath_comp...Out_reg[27]/D	0.483	0.292	0.191	0.0	CLK	CLK		
Path 16	0.378	1	2	Datapath_comp...Out_reg[30]/C	Datapath_comp...Out_reg[31]/D	0.483	0.292	0.191	0.0	CLK	CLK		
Path 17	0.391	1	2	Datapath_comp...Out_reg[29]/C	Datapath_comp...Out_reg[29]/D	0.496	0.249	0.247	0.0	CLK	CLK		
Path 18	0.398	1	2	Datapath_comp...Out_reg[14]/C	Datapath_comp...Out_reg[14]/D	0.503	0.256	0.247	0.0	CLK	CLK		
Path 19	0.398	1	2	Datapath_comp...Out_reg[22]/C	Datapath_comp...Out_reg[22]/D	0.503	0.256	0.247	0.0	CLK	CLK		
Path 20	0.398	1	2	Datapath_comp...Out_reg[10]/C	Datapath_comp...Out_reg[10]/D	0.503	0.256	0.247	0.0	CLK	CLK		

7-2. Verification of Correct Processor Design and Operation



7-2.1 Assembly Test Program

The above assembly program verifies the correct operation as follows: The `ADDI` instructions are used to insert initial values into the registers. This can be verified via the `ALUResult` and the `Result` of the processor. This also confirms the proper functioning of the extend unit for the second ALU operand.

Next, by executing the `ADD` instruction, we can check if the values from `ADDI` were stored correctly in the registers and if the `ADD` operation works correctly. At this point, we also confirm the correct operation of the `ALUSrc` multiplexer, which previously (in immediate instructions) selected the second ALU input from the Extend unit, but now must select from the RegisterFile to function correctly. Additionally, we observe the proper update of `RegWrite` by the control unit.

Similar checks are achieved with the remaining R-type instructions, which are executed to verify the results of all operations. In the `SUB` instruction, we check both the Negative flag and the

Zero flag (instruction 14). The V (Overflow) flag was difficult to verify because it would require an `ADDI` with a very large constant number, and since we lack the `LUI` instruction in our set, it would have to be called iteratively. Therefore, I restricted the overflow flag verification to the ALU testbench alone.

With the `XORI`, `ANDI`, `ORI`, and `XOR`, `AND`, `OR` instructions, we verify the correct operation results, meaning the control unit properly decodes and sends the correct signals to the datapath.

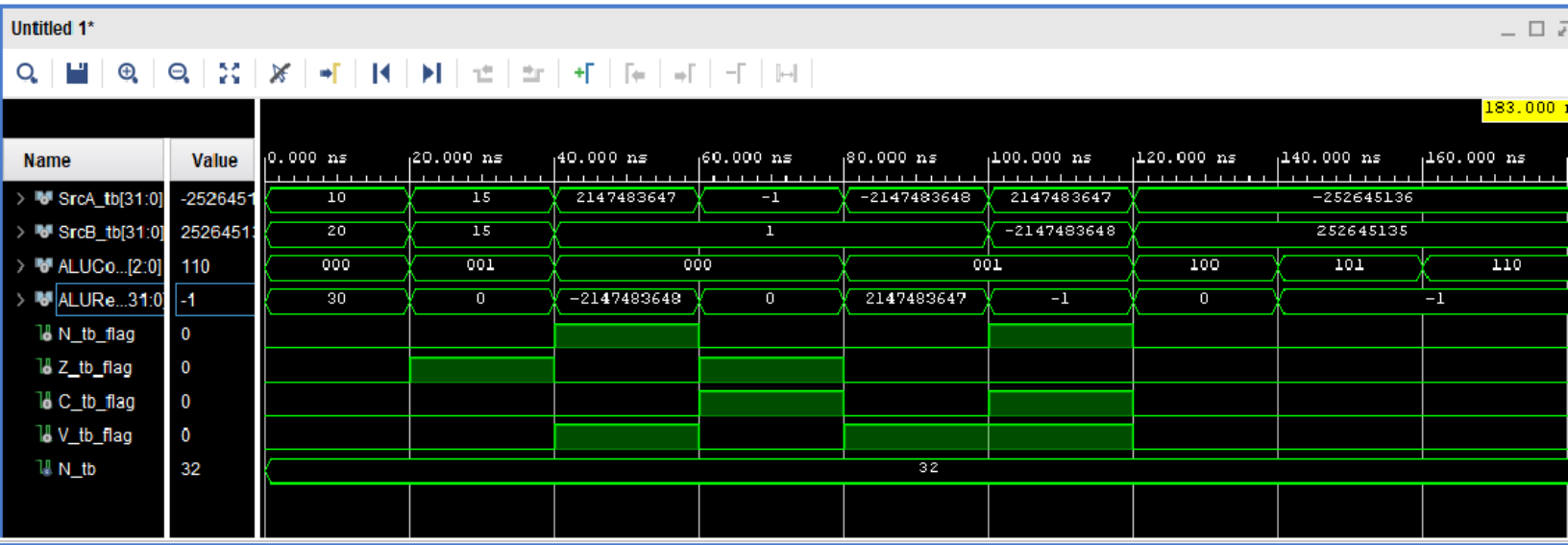
Finally, we test the `LW` and `SW` instructions. First, we load a value into memory position 4 and then into position 8. The `Store` operation itself can be checked via the `ALUResult`; this tells us if the Extend was done correctly (and thus if the correct `ImmSrc` signal arrived from the control unit) and if the `ALUSrc` signal was updated correctly so that the `ALUResult` is the intended address. Furthermore, we can monitor `WriteData` to see if the data we intend to write is actually routed to the RAM. To verify if it was truly written to RAM, since we don't have access to the memory or the `MemWrite` signal in the implemented design, we can execute an `LW` and check if we get the expected value. This happens in the last two instructions, where I specifically load from position 4 right after overwriting the old value, to confirm it updated in time.

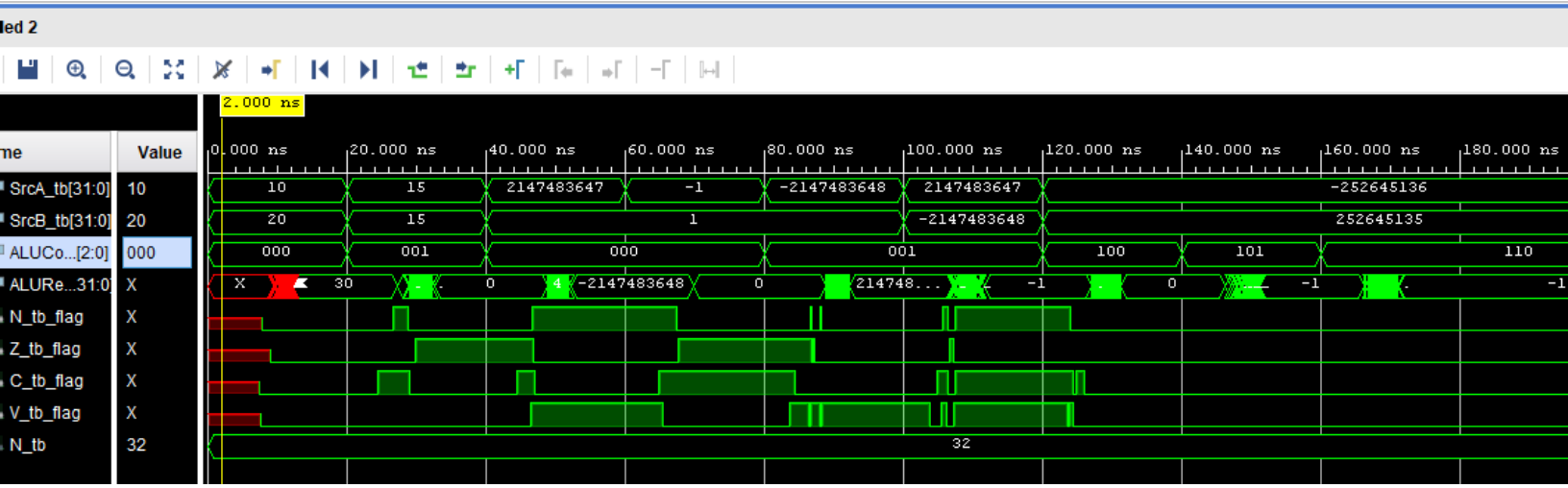
Simultaneously, we can observe which instruction is executing via the `Instr_out` output of the processor and whether the Program Counter is updating correctly.

7-2.2 ALU Timing Diagrams

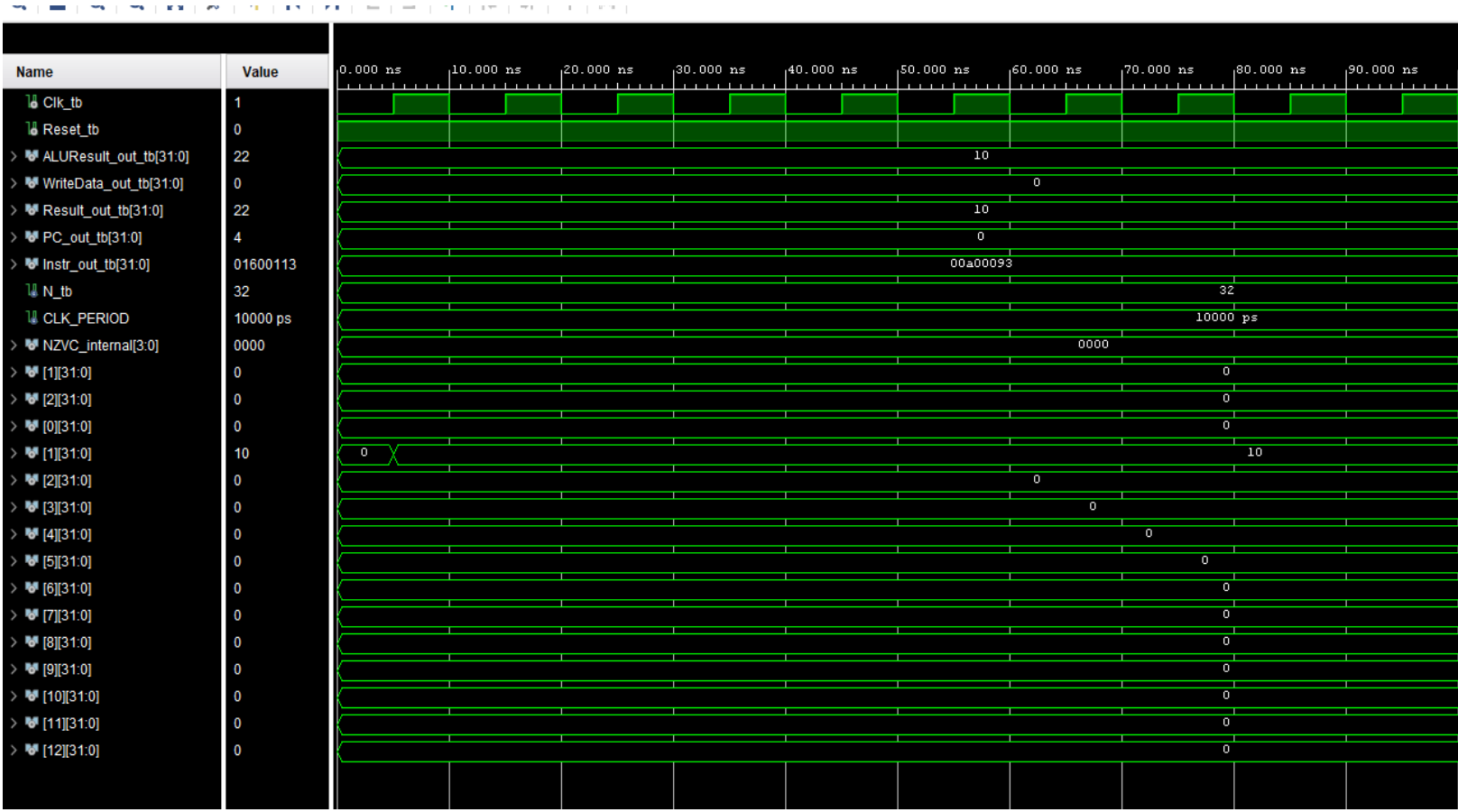
Below, we see the execution of the behavioral and Post-Implementation Timing simulation of the ALU. The operations tested in sequence are: `ADD` (ALUControl = 000), `SUB` (ALUControl = 001) with Z=1 flag, `ADD` with overflow, `ADD` with carry and zero flag, and `SUB` with overflow (both positive and negative overflow). Finally, the `AND`, `OR`, and `XOR` operations are tested where all flags are 0 and the results should be 0 for `AND` and FFFFFFFF (-1) for the other two, given the hexadecimal inputs FOFOFOFO and OFOFOFOF.

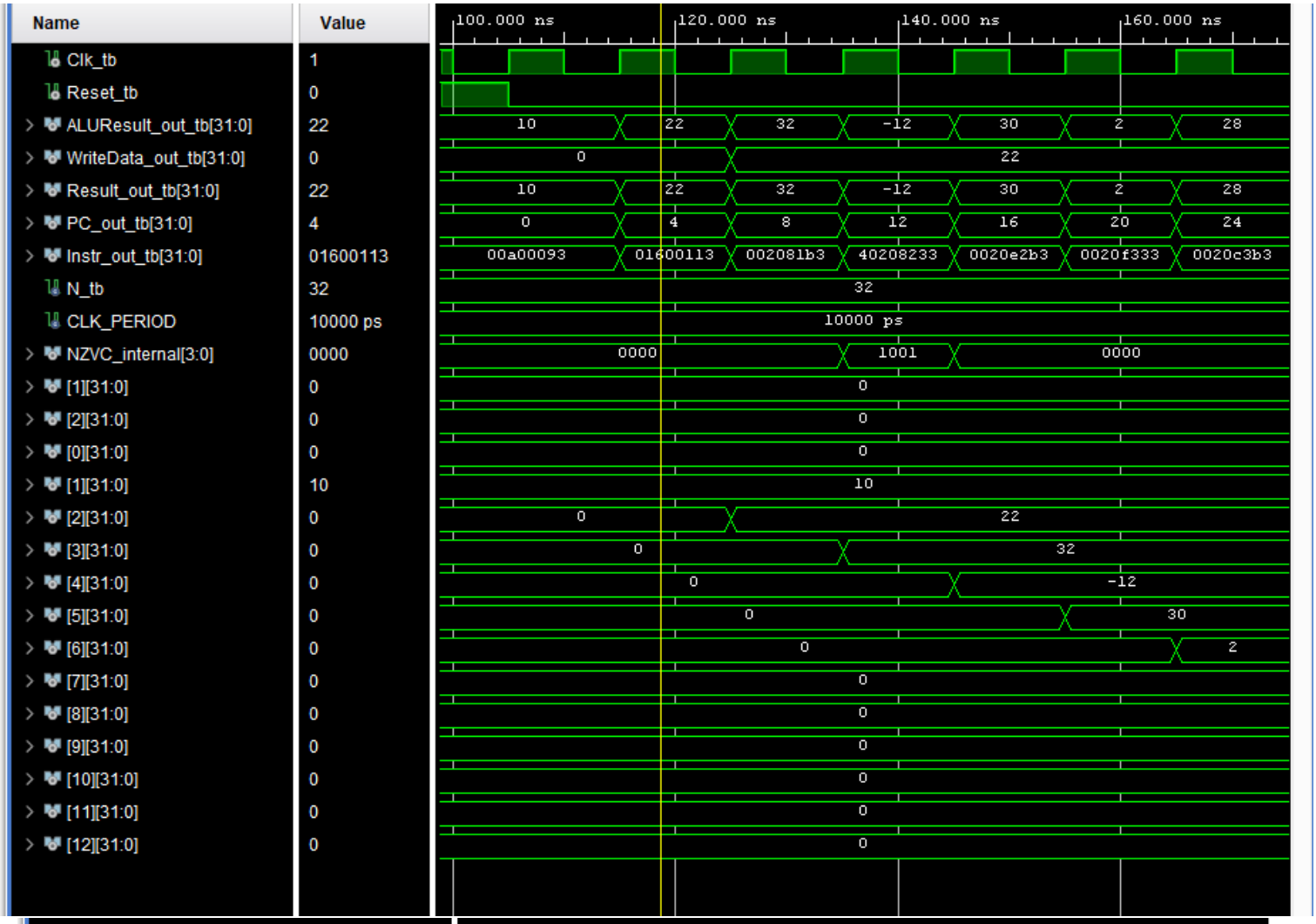
In the Post-Implementation Timing simulation, we observe real delays and some glitches, which are expected since updates do not occur instantaneously as they do in the ideal behavioral simulation. The crucial point is that values update and stabilize before the next value change. I change values every 20 ns because the propagation delays of the ALU as a standalone unit are ~15 ns, so a 10 ns period would cause issues.

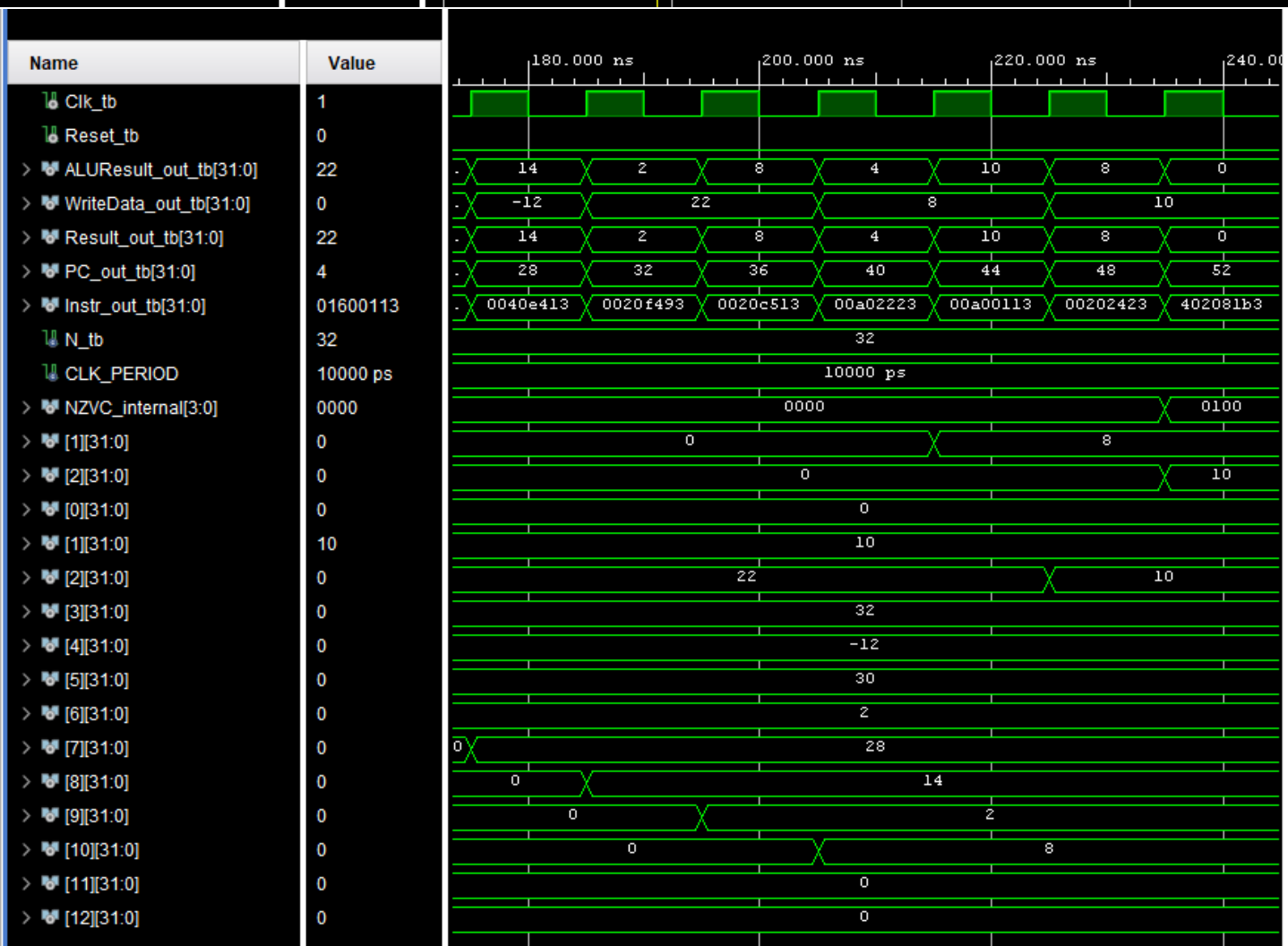


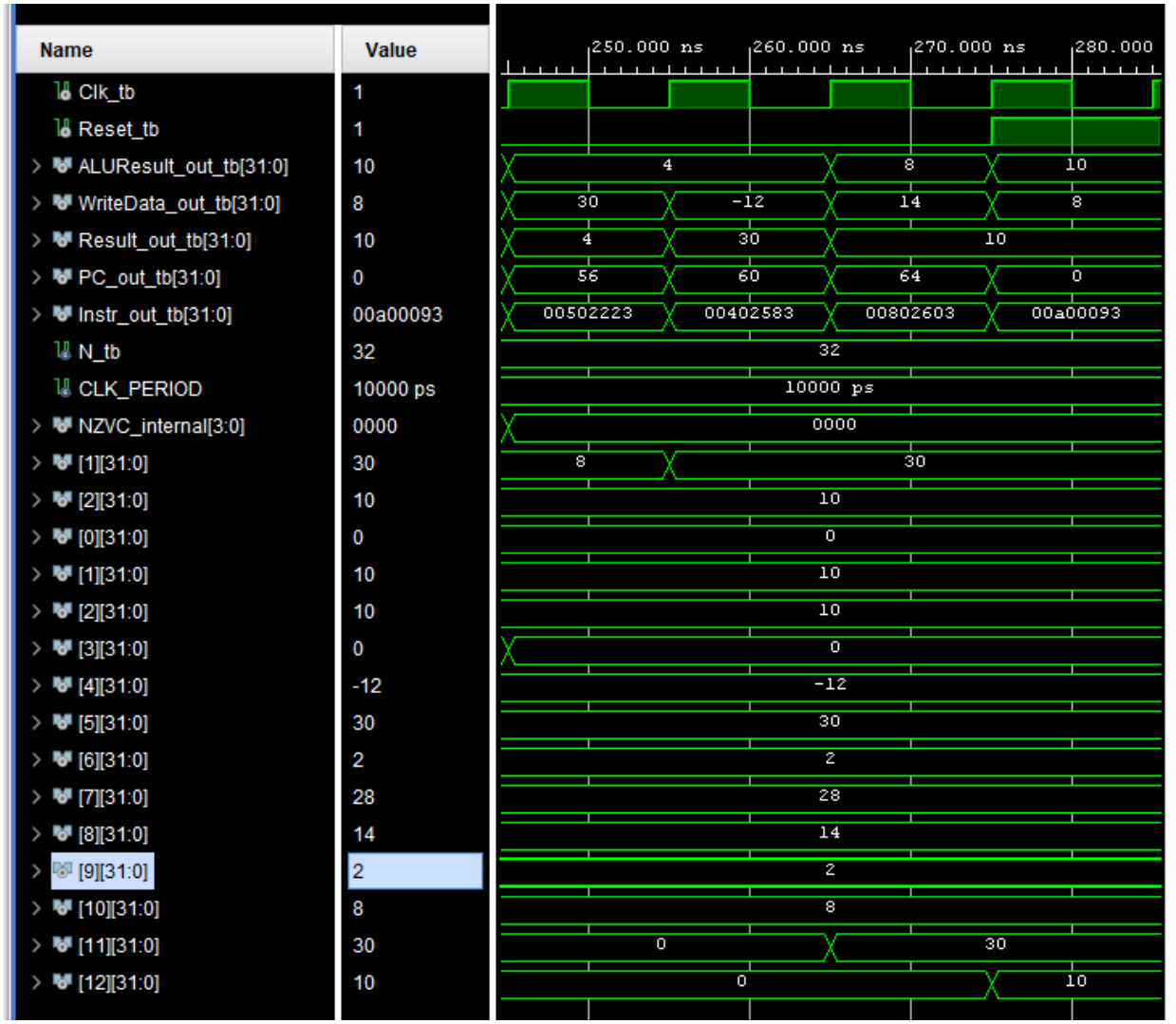


7-2.3 Behavioral Simulation for the Complete Processor




Name
Value
100.000 ns
120.000 ns
140.000 ns
160.000 ns
Clk_tb 1
Reset_tb 0
ALUResult_out_tb[31:0] 22 10 22 32 -12 30 2 28
WriteData_out_tb[31:0] 0 0 22
Result_out_tb[31:0] 22 10 22 32 -12 30 2 28
PC_out_tb[31:0] 4 0 4 8 12 16 20 24
Instr_out_tb[31:0] 01600113 00a00093 01600113 002081b3 40208233 0020e2b3 0020f333 0020c3b3
N_tb 32 32
CLK_PERIOD 10000 ps 10000 ps
NZVC_internal[3:0] 0000 0000 1001 0000
[1][31:0] 0 0
[2][31:0] 0 0
[0][31:0] 0 0
[1][31:0] 10 10
[2][31:0] 0 0 22
[3][31:0] 0 0 32
[4][31:0] 0 0 -12
[5][31:0] 0 0 30
[6][31:0] 0 0 2
[7][31:0] 0 0
[8][31:0] 0 0
[9][31:0] 0 0
[10][31:0] 0 0
[11][31:0] 0 0
[12][31:0] 0 0


Name
Value
180.000 ns
200.000 ns
220.000 ns
Clk_tb 1
Reset_tb 0
ALUResult_out_tb[31:0] 22 14 2 8 4 10 8 0
WriteData_out_tb[31:0] 0 -12 22 8 10
Result_out_tb[31:0] 22 14 2 8 4 10 8 0
PC_out_tb[31:0] 4 28 32 36 40 44 48 52
Instr_out_tb[31:0] 01600113 0040e413 0020f493 0020c513 00a02223 00a00113 00202423 402081b3
N_tb 32 32
CLK_PERIOD 10000 ps 10000 ps
NZVC_internal[3:0] 0000 0000 0100
[1][31:0] 0 0 8
[2][31:0] 0 0 10
[0][31:0] 0 0
[1][31:0] 10 10
[2][31:0] 0 22 10
[3][31:0] 0 32
[4][31:0] 0 -12
[5][31:0] 0 30
[6][31:0] 0 2
[7][31:0] 0 0 28
[8][31:0] 0 0 14
[9][31:0] 0 0 2
[10][31:0] 0 0 8
[11][31:0] 0 0
[12][31:0] 0 0



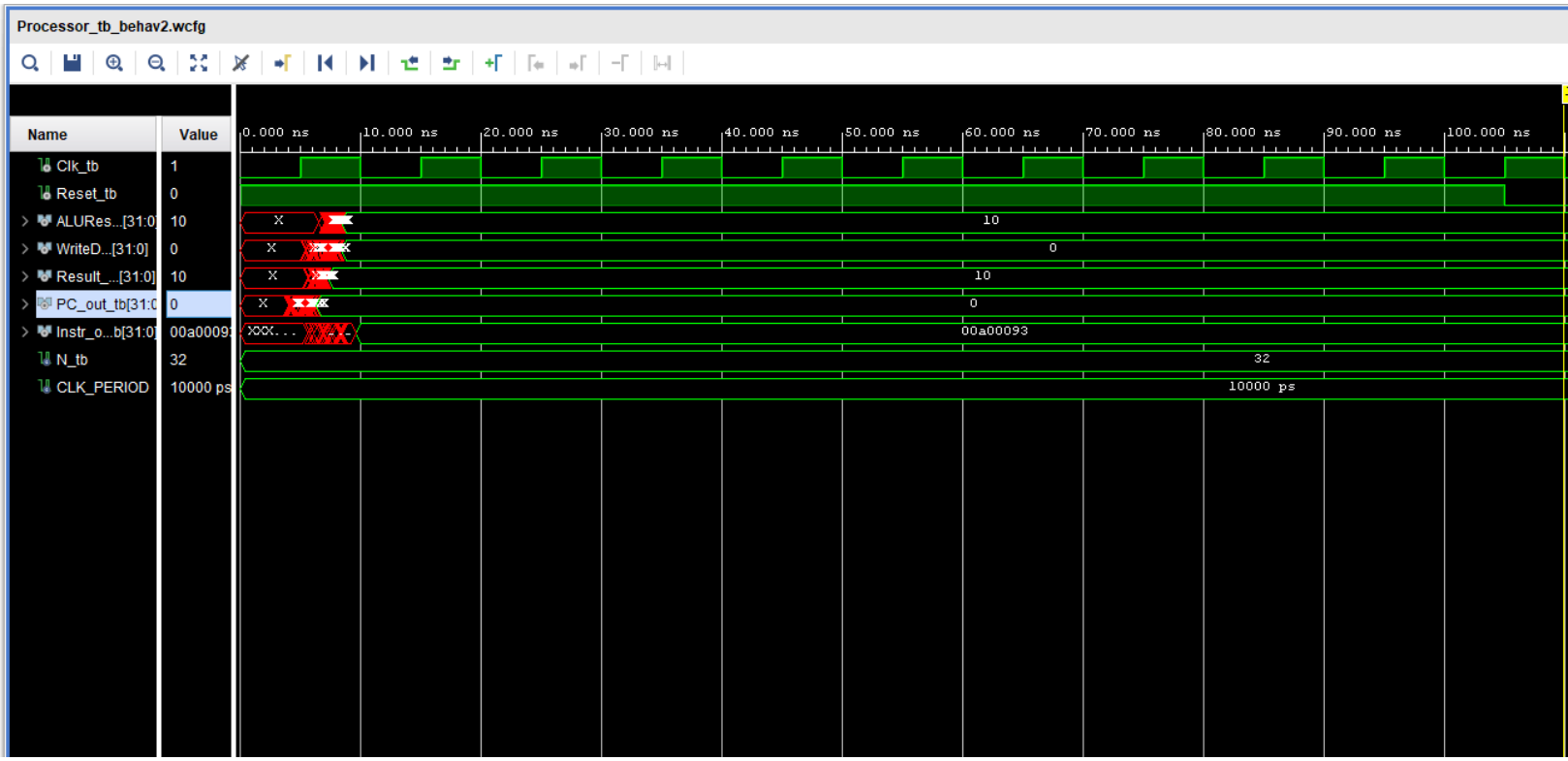
In the processor's timing diagrams (behavioral model), we observe the following results for the operations mentioned in 7.2.1: Initially, for the first 100 ns, Reset is set to 1 to ensure all system components enter their initial state (Global reset). Following this, we see that throughout the simulation, the Program Counter increments by 4 as expected, and the instructions (`Instr_out`) are loaded sequentially.

Specifically, during the first two operations (`ADDI`), we observe the value in `ALUResult` and `Result`, while `WriteData` is irrelevant because the corresponding `MemWrite` signal is 0 from the Control unit. Subsequently, in the rest of the R-type and I-type operations, the results match the expected values commented next to each assembly instruction.

The `SW` then stores the correct value (`WriteData` = 8) in the correct location (`ALUResult` = 4). The next `ADDI` behaves like the previous ones, and the `SW` stores the value 10 in position 8. We then have a subtraction that correctly triggers the Zero = 1 flag. Finally, we execute another `SW` to memory location 4 (which already contained data) to verify if the subsequent `LW` will work correctly. The value is stored properly, and the last two `LOAD` instructions fetch the expected values into the `Result` field (since `ALUResult` holds the memory address for LOAD).

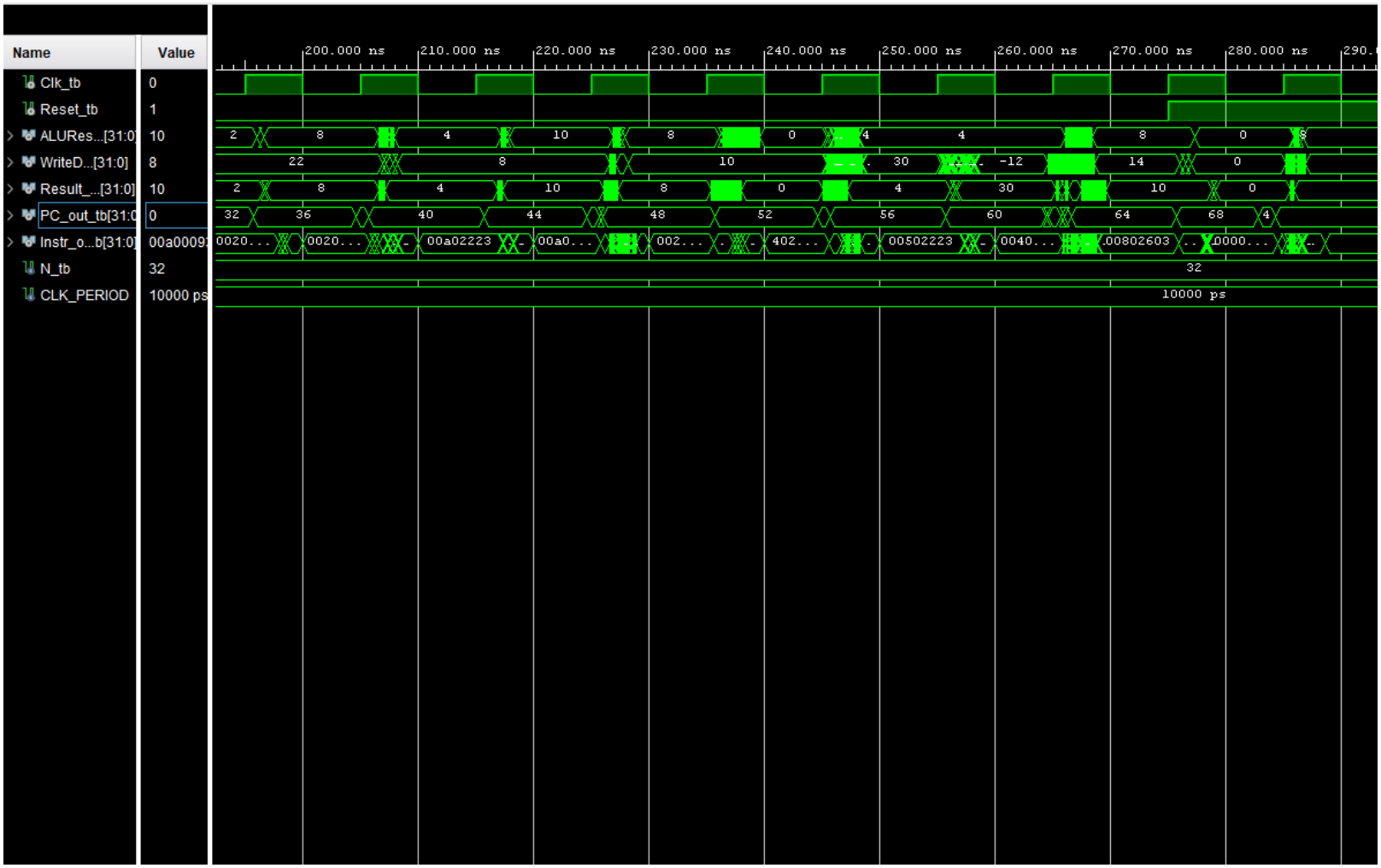
Because this is a behavioral simulation, we have access to internal signals, memory locations, and registers, allowing us to monitor updates there. Specifically, the signals added under `NZVC_internal` are: data memory locations (positions 4 and 8) and the first 12 registers used in the assembly program. We observe that the register file update occurs on the rising edge of the clock at the end of the execution cycle (i.e., the beginning of the next one). Thus, the new value becomes available for reading at the start of the next clock cycle via an asynchronous read.

Post-Implementation Timing Simulation



Processor_tb_behav2.wcfg

Processor_tb_behav2.wcfg



The same behavior is observed in the Post-Implementation Timing simulation, with the only difference being the glitches present until the outputs stabilize, and the real circuit delays. Nevertheless, there are no issues since the period is set to 10 ns and values change every 10 ns, thus respecting the timing constraints from step 7.1.4. Also, we lack access to internal signals because the synthesis tool has merged signals and generated an optimized netlist, making it difficult to trace specific wires back to internal variables. The outputs confirm the correctness established in the behavioral simulation.

7-3. Analysis of Processor Synthesis and Implementation Results

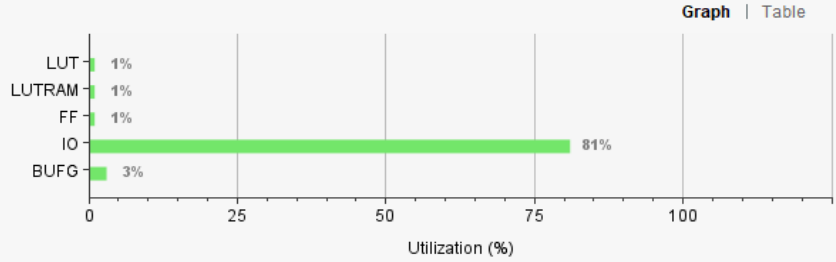
7-3.1 Utilization After **Synthesis**

Hierarchy

Name	Slice LUTs (53200)	Slice Registers (106400)	Bonded IOB (200)	BUFGCTRL (32)
Processor	272	30	162	1
Datapath_component (Datapath)	272	30	0	0
ALU_component (ALU)	0	0	0	0
DataMemory_component (DataM	32	0	0	0
PC_component (Program_coun	158	30	0	0
RegisterFile_component (Regis	82	0	0	0

Utilization after **implementation**

Utilization — Post-Synthesis | **Post-Implementation** — Graph | Table



Power

Total On-Chip Power:	**0.148 W**
Junction Temperature:	**26.7 °C**
Thermal Margin:	58.3 °C (4.9 W)
Effective ϑJA:	11.5 °C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Medium

Implemented Power Report

ign Runs | **Utilization** | DRC | Methodology | Power

Hierarchy

Name	Slice LUTs (53200)	Slice Registers (106400)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Bonded IOB (200)	BUFGCTRL (32)
Processor	250	30	82	174	76	162	1
Datapath_component (Datapath)	250	30	82	174	76	0	0
ALU_component (ALU)	0	0	8	0	0	0	0
DataMemory_component (DataM	32	0	8	0	32	0	0
PC_component (Program_coun	157	30	62	157	0	0	0
RegisterFile_component (Regis	62	0	28	18	44	0	0

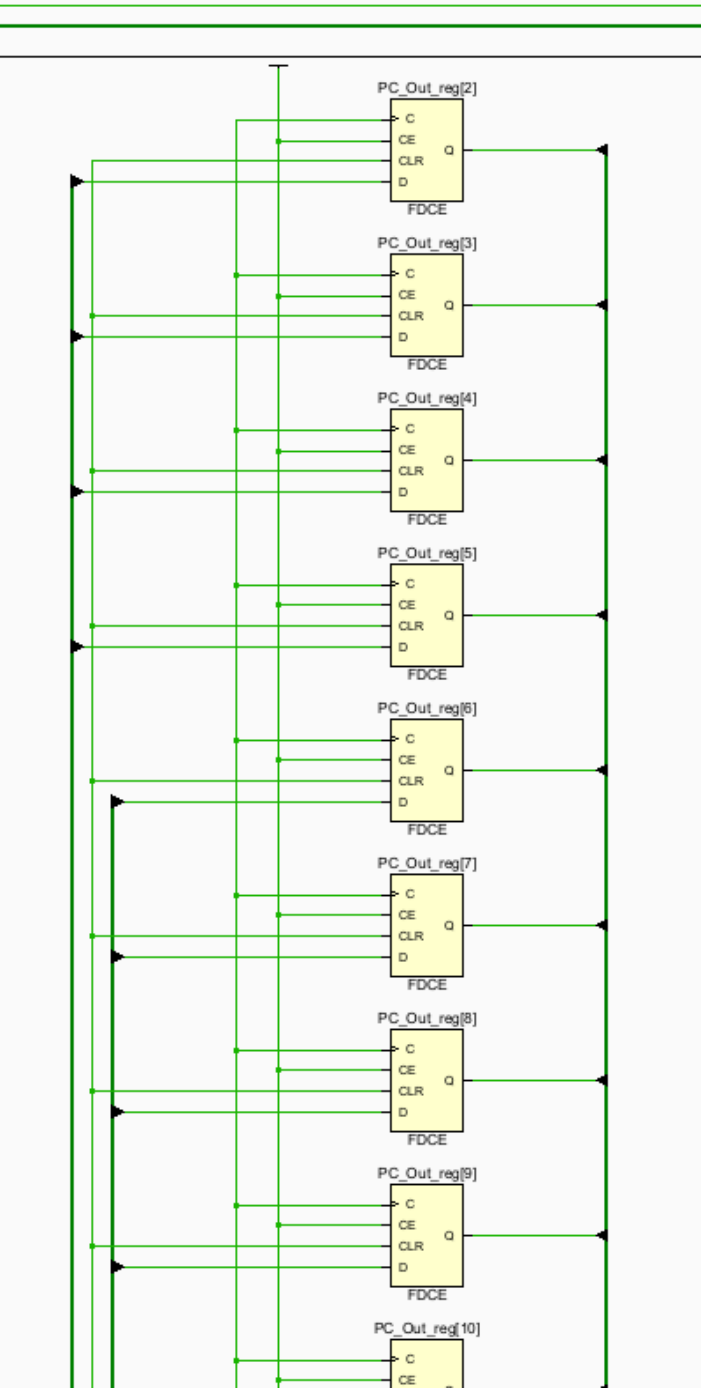
Primitives

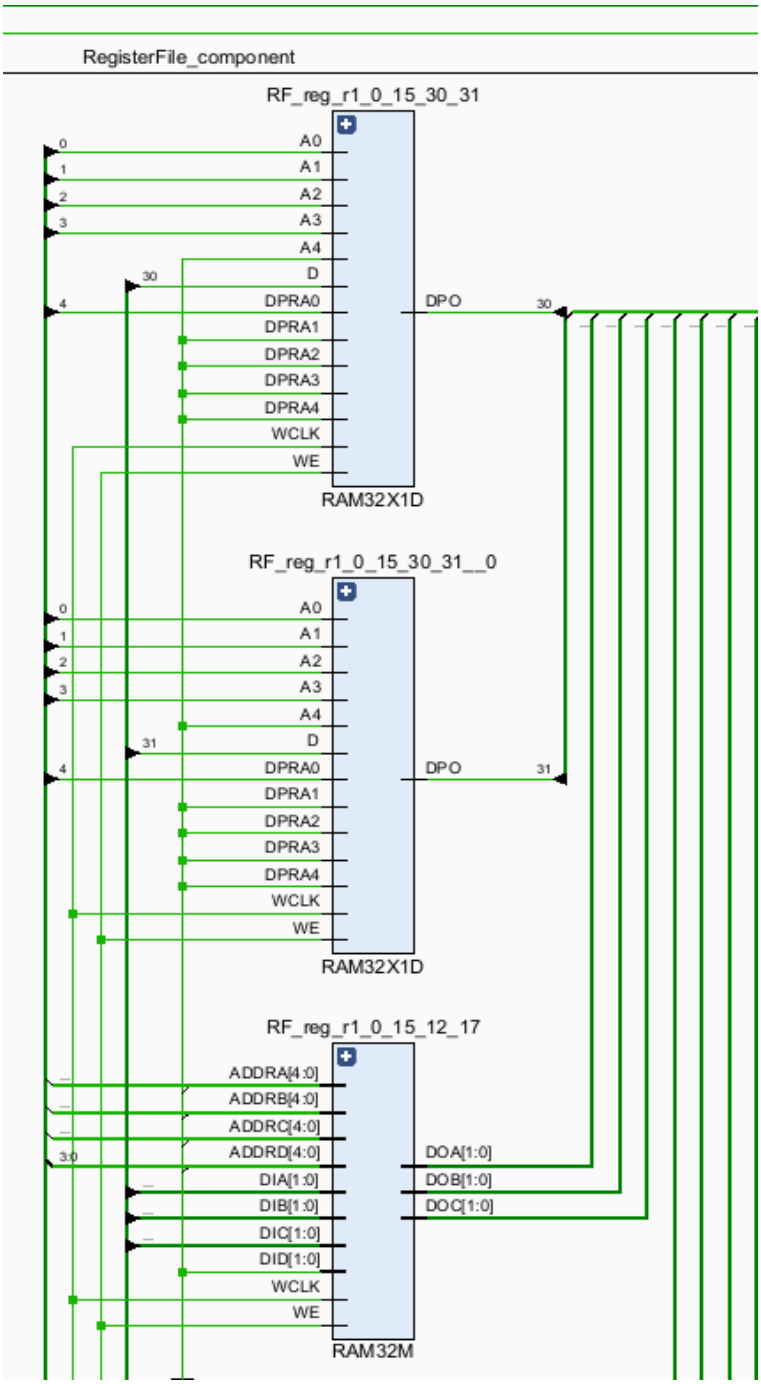
Ref Name	Used	Functional Category
OBUF	160	IO
LUT6	74	LUT
RAMD32	68	Distributed Memory
LUT5	61	LUT
LUT3	58	LUT
RAMS32	52	Distributed Memory
LUT2	34	LUT
FDCE	30	Flop & Latch
CARRY4	16	CarryLogic
LUT4	5	LUT
IBUF	2	IO
LUT1	1	LUT
BUFG	1	Clock

It is clear that Latches are not used.

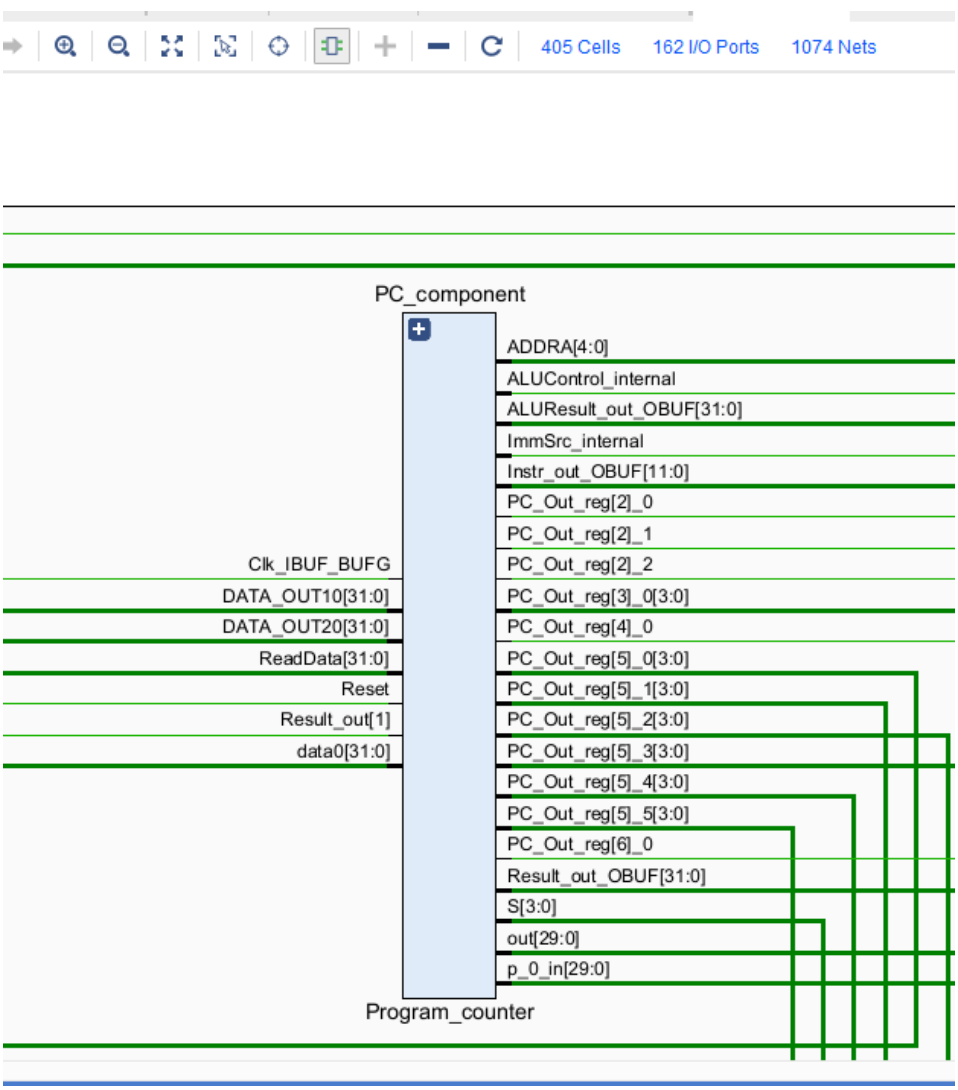
Observations from the Implemented Design:

- **250 LUTs utilized.** Out of these, 174 were used as Logic to implement combinational logic (ALU, Multiplexers, Control Unit). The remaining 76 were used as Memory (LUTRAM) for Distributed RAM, not Block RAMs.
- The presence of 16 CARRY4 units confirms the implementation of 32-bit adders for the ALU and the PC calculation (8 CARRY4 per 32-bit adder).
- **30 Flip-Flops.** This number corresponds exclusively to the Program Counter (PC) register. Since instructions are 32-bit (4 bytes) and addressing is aligned, the 2 least significant bits of the PC are always 0 and do not need to be stored in physical Flip-Flops, hence 30 instead of 32 FFs.
- The Register File was not implemented with Flip-Flops but as Distributed RAM (LUTRAM), saving significant register resources. No Latches were used.





The photos above show the PC FFs and the RAM implementation of the Register Files. In both reports (Synthesis & Implementation), the Control Unit does not appear as a separate entity in the hierarchy. Because the Control Unit consists entirely of combinational logic, the synthesis tool integrated its logic directly into the `Datapath_component`.

Another observation is the absence of ROM blocks for the Instruction Memory. However, the Synthesis Log indicates no undesired simplifications occurred: `INFO: [Synth 8-638] synthesizing module 'Instruction_memory'... INFO: [Synth 8-256] done synthesizing module 'Instruction_memory'`

The Instruction Memory was implemented as combinational logic using Logic LUTs rather than dedicated memory blocks. Consequently, its resources are merged into the general logic of the `Datapath_component`. This integration is confirmed by the increased LUT count in the `PC_component` (157 LUTs), indicating that the ROM's combinational logic merged with the PC logic. Furthermore, its correct functionality is validated by the successful execution of the program in the simulation. We observe that the `PC_component` directly drives the `Instr_out` output signals. This proves that the combinational logic mapping addresses to instructions (i.e., the instruction memory) has been embedded within the `PC_component`, justifying the increased LUT usage.